Overview of the Dynblend Package

https://gemini.google.com/app/b09bd612a0a4fcb1

User prompt: What is the f(kl distance) in the pi formula above

Response: The f(KL distance) term acts as the **squared bias penalty** in the optimization. It mathematically enforces the trade-off between statistical uncertainty and clinical divergence.

If we derive the optimal blending weight by minimizing the expected loss (Mean Squared Error) of the blended survival curve, we must account for both Variance and Bias (MSE=Variance+Bias$_2$).

Assuming your external population curve S_{ext} represents the long-term "ground truth," the bias of relying on your extrapolated trial curve is the difference between them: $\Delta(t)=S_{obs}(t)-S_{ext}(t)$.

When you calculate the total MSE for your blended curve $S_{blend}=\pi S_{obs}+(1-\pi)S_{ext}$ and minimize it with respect to π, the analytical solution is:

$$\pi(t)=V_{obs}(t)+V_{ext}(t)+\Delta(t)_2 V_{ext}(t)$$

The $\Delta(t)_2$ term is exactly where the Kullback-Leibler divergence enters. By applying a second-order Taylor series expansion, the KL divergence between two Bernoulli distributions (your two survival probabilities at time t) is asymptotically proportional to their squared difference weighted by the Fisher Information:

$$D_{KL}(S_{obs}||S_{ext})\approx 2S_{ext}(1-S_{ext})(S_{obs}-S_{ext})_2$$

Therefore, the squared difference $\Delta(t)_2$ is simply a scaled version of your point-wise KL divergence:

$$\Delta(t)_2\approx 2\times S_{ext}(1-S_{ext})\times D_{KL}(S_{obs}||S_{ext})$$

How it controls the blend: This creates a dynamic tug-of-war in the denominator of the $\pi(t)$ equation.

1. If the trial extrapolation closely matches the external population, the KL distance is near 0. The formula defaults to pure inverse-variance weighting.
2. If the trial extrapolation drifts wildly away from the external population, the KL distance explodes. This heavily penalises the denominator.
3. To maintain a high $\pi(t)$ weight when the KL distance is large, the trial data must possess overwhelming precision (V_{obs} must be virtually zero) to justify the divergence.
4. As soon as the trial's extrapolated variance V_{obs} inevitably increases, it can no longer overpower the KL distance penalty, and $\pi(t)$ collapses to 0.

This means f(KL distance) is the mechanism that says: *"I will only let you deviate from the population baseline if you have overwhelming statistical proof. The moment your proof (precision) drops, I am pulling*

you back to the baseline."

User prompt: So can this kl term in the denominator be substituted for another measure like the wassetstein

Response: Yes, absolutely. In fact, substituting the KL divergence for the Wasserstein distance (often called the Earth Mover's Distance) is not just possible—it might actually be a much more robust and mathematically elegant choice for your dynblend package.

Because we are evaluating the distance point-wise at time t (where the state is a Bernoulli distribution: alive with probability S, dead with probability $1-S$), the math simplifies beautifully under the Wasserstein metric.

The Wasserstein Distance for Survival For two discrete Bernoulli distributions, the 1-Wasserstein distance (W_1) is simply the absolute difference between their probabilities:

$$W_1(S_{obs}(t), S_{ext}(t)) = |S_{obs}(t) - S_{ext}(t)|$$

This means that the squared bias term $\Delta(t)^2$ in your denominator is literally just the squared 1-Wasserstein distance:

$$\Delta(t)^2 = W_1^2$$

You can rewrite the automated blending weight directly in terms of Wasserstein distance without needing any Taylor series approximations:

$$\pi(t) = V_{obs}(t) + V_{ext}(t) + W_1(S_{obs}(t), S_{ext}(t))^2 V_{ext}(t)$$

Why Wasserstein might be superior to KL for your package:

1. **It is symmetric:** KL divergence is asymmetric—KL($Obs||Ext$) is different from KL($Ext||Obs$). This forces you to make a subjective choice about which curve is the "reference" or use a symmetrised version (like Jeffreys divergence). Wasserstein is perfectly symmetric, treating the distance between the two curves identically.
2. **It doesn't blow up at the tails (The log(0) problem):** If one of your extrapolated curves drops to exactly $S(t)=0$ or 1, the log components of KL divergence shoot to infinity. This can crash the blending algorithm deep into the extrapolation horizon. The Wasserstein distance is strictly bounded between 0 and 1, remaining completely stable no matter what the survival probabilities do at the tail end of the model.
3. **It maps perfectly to Mean Squared Error:** Because W_1^2 is exactly the squared error $(S_{obs} - S_{ext})^2$, using it directly optimizes the true Bias-Variance tradeoff of the blended curve without relying on Fisher Information scaling factors.

If you implement this in dynblend, you are essentially automating the blend by minimizing the Wasserstein-penalised expected loss. It tells the algorithm: *"Weight the curves by their precision, but penalize the trial data precisely by the amount of probability mass (W_1) you would have to physically move to align it with the population baseline."*

User prompt: What else can we use. Jenson Shannon etc

Response: Yes, Jensen-Shannon Divergence (JSD) is an excellent candidate and solves the two biggest mathematical flaws of using standard Kullback-Leibler divergence for survival extrapolation: asymmetry and the log(0) tail explosion.

Because you are evaluating the distance point-wise at time t over a binary state (alive with probability $S(t)$, dead with probability $1-S(t)$), several information-theoretic and probability metrics become highly tractable for your denominator penalty term.

Here is how Jensen-Shannon and other key alternatives operate as the f(distance) penalty:

1. Jensen-Shannon Divergence (JSD)

JSD smooths the KL divergence by comparing both survival curves to a shared midpoint (a mixture distribution), rather than comparing one directly to the other.

- **The Mixture:** $M(t) = \frac{1}{2}S_{obs}(t) + \frac{1}{2}S_{ext}(t)$
- **The Calculation:** $JSD(t) = \frac{1}{2}D_{KL}(S_{obs} \,||\, M) + \frac{1}{2}D_{KL}(S_{ext} \,||\, M)$

Why it works well:

- **Symmetry:** $JSD(Obs, Ext) = JSD(Ext, Obs)$. You no longer have to justify which curve is the "reference" baseline.
- **Strictly Bounded:** Unlike KL which shoots to infinity if a curve hits 0 or 1, JSD is strictly bounded between 0 and ln(2) (or 1 if using base-2 logs). If your trial curve drops to exactly 0 survival late in the horizon but the external curve is at 0.05, KL crashes your algorithm; JSD safely registers a finite maximum penalty.

2. Hellinger Distance

Hellinger distance is frequently used in asymptotic statistics as a more stable, symmetric alternative to KL divergence. Instead of using logarithms (which cause the tail instability), it relies on square roots.

- **The Calculation (Bernoulli):**
  $H^2(t) = \frac{1}{2}\left[(\sqrt{S_{obs}(t)} - \sqrt{S_{ext}(t)})^2 + (\sqrt{1-S_{obs}(t)} - \sqrt{1-S_{ext}(t)})^2\right]$

Why it works well:

- Like JSD, it is symmetric and strictly bounded (between 0 and 1).
- Because it maps closely to the Bhattacharyya coefficient (measuring the exact geometric overlap of the two distributions), it provides a very smooth gradient. The penalty increases steadily as the curves diverge, making the automated $\pi(t)$ transition highly predictable.

3. Total Variation Distance (TVD)

TVD measures the maximum possible difference between the probability assignments of two distributions.

- **The Calculation:** $TVD(t) = _{21}\sum |p_i - q_i|$

Why it works well (and a mathematical shortcut): For a discrete binary outcome like point-wise survival, the TVD formula simplifies to:

$TVD(t) = 21(|S_{obs} - S_{ext}| + |(1 - S_{obs}) - (1 - S_{ext})|)$

Which collapses entirely to:

$TVD(t) = |S_{obs}(t) - S_{ext}(t)|$

Notice that for this specific point-wise Bernoulli application, **Total Variation Distance is mathematically identical to the 1-Wasserstein distance.** If you implement the absolute difference $|S_{obs} - S_{ext}|$ as your penalty term, you can correctly justify it in your documentation as either the Wasserstein distance or the TVD.

Implementing in the Formula

If you substitute any of these into your automated blending weight, you replace the squared bias term:

$\pi(t) = V_{obs}(t) + V_{ext}(t) + \lambda \cdot \text{Distance}(t)_2 V_{ext}(t)$

(Note: You may need a scaling hyperparameter λ when swapping metrics, as a metric bounded by ln(2) like JSD will exert a different magnitude of penalty on the denominator than the unbounded Fisher-scaled KL divergence, altering how aggressively the algorithm switches to the external curve).

User prompt: So these are blending comparing s external and observed directly? What about via the third internodiated blended curve

Response: You have just hit on the fundamental difference between calculating a **penalty** (direct comparison) and calculating a **barycenter** (intermediated comparison).

When you introduce the intermediated third curve (S_{blend}), you are no longer just measuring how far apart the two source curves are to penalize the variance. Instead, you are setting up an optimization problem to find the "center of mass" between them.

In mathematics, the optimal intermediated curve that minimizes the weighted sum of distances to two source distributions is called a **Barycenter** (or Fréchet mean).

Here is how the choice of distance metric completely defines the intermediated curve, and how it magically leads right back to the automated $\pi(t)$ formula.

1. The Intermediated Objective

You want to find the curve S_{blend} that minimizes the expected information loss to both sources, weighted by

$\pi(t)$ and $1-\pi(t)$. The objective function is:

$J(S_{blend}) = \pi \cdot \text{Distance}(S_{obs}, S_{blend}) + (1-\pi) \cdot \text{Distance}(S_{ext}, S_{blend})$

Depending on the distance metric you plug into this intermediated equation, the optimal S_{blend} changes:

- **If Distance = Forward KL:** The solution (as we derived earlier) is the arithmetic mean. $S_{blend} = \pi S_{obs} + (1-\pi) S_{ext}$.
- **If Distance = Reverse KL:** The solution is the geometric mean. $S_{blend} \propto S_{obs}^{\pi} \times S_{ext}^{1-\pi}$.
- **If Distance = Wasserstein (W_1):** Finding the Wasserstein barycenter of two Bernoulli distributions yields the exact same result as Forward KL. The optimal transport plan proves that $S_{blend} = \pi S_{obs} + (1-\pi) S_{ext}$.

This is a massive theoretical justification for your package: **Linear pooling is the mathematically optimal intermediated curve under both Kullback-Leibler and Wasserstein frameworks.**

2. Automating $\pi(t)$ via the Intermediated Curve

If S_{blend} is our intermediated curve, how does this help us automate the estimation of $\pi(t)$?

We do this by calculating the **Mean Squared Error (MSE) of the intermediated curve itself**, and finding the $\pi(t)$ that minimizes it.

MSE is Variance + Bias^2. If we define our intermediated curve as $S_{blend} = \pi S_{obs} + (1-\pi) S_{ext}$, we can break down its two components:

The Variance of the Intermediated Curve: Assuming the trial and external data are independent, the variance of your blended curve is:

$V_{blend} = \pi_2 V_{obs} + (1-\pi)_2 V_{ext}$

The Bias of the Intermediated Curve: Assuming the massive external registry (S_{ext}) represents the asymptotic "truth," the bias of your blended curve is the distance from the blend to the truth:

$\text{Bias} = S_{blend} - S_{ext}$

Substitute the linear pool definition of S_{blend}:

$\text{Bias} = [\pi S_{obs} + (1-\pi) S_{ext}] - S_{ext}$
$\text{Bias} = \pi(S_{obs} - S_{ext})$

Notice what happened here: the bias of the intermediated curve is simply π multiplied by the direct distance ($\Delta = S_{obs} - S_{ext}$).

Minimizing the Total Error: The total expected loss (MSE) of the intermediated curve is:

$\text{MSE}(S_{blend}) = \pi_2 V_{obs} + (1-\pi)_2 V_{ext} + \pi_2 (S_{obs} - S_{ext})_2$

To find the optimal π that automates the blend, we take the derivative of this MSE with respect to π and set

it to zero:

$$\partial \pi \partial \text{MSE} = 2\pi V_{obs} - 2(1-\pi)V_{ext} + 2\pi(S_{obs} - S_{ext})_2 = 0$$

Divide by 2 and expand:

$$\pi V_{obs} - V_{ext} + \pi V_{ext} + \pi(S_{obs} - S_{ext})_2 = 0$$

Factor out π:

$$\pi[V_{obs} + V_{ext} + (S_{obs} - S_{ext})_2] = V_{ext}$$

Solve for π:

$$\pi(t) = V_{obs}(t) + V_{ext}(t) + (S_{obs}(t) - S_{ext}(t))_2 V_{ext}(t)$$

The Grand Unification

By setting up the problem via the intermediated curve S_{blend}, everything unifies perfectly.

The squared term in the denominator—$(S_{obs} - S_{ext})_2$—is exactly the squared Wasserstein distance (W_{12}), which is also the Taylor-series equivalent of the Kullback-Leibler divergence.

You don't have to choose between directly comparing the source curves OR using an intermediated curve. When you optimize the expected loss of the intermediated curve, the direct distance between the two sources organically falls out of the derivative and becomes the exact penalty term that drives $\pi(t)$ to zero.

User prompt: So you're saying the barycent and speedometer approaches five the same optimal solution?

Response: Yes, exactly. (Assuming "speedometer" is an autocorrect for the "squared error" or "direct comparison" approach!)

Both philosophical starting points lead you to the exact same optimal formula for $\pi(t)$. This gives your methodology in dynblend a massive theoretical advantage, because you can justify the exact same R code from two completely different statistical paradigms:

1. The Information Theory Justification (Direct Comparison) You argue that the algorithm should weight the curves by their precision, but explicitly penalize the trial data based on its information divergence (KL or Wasserstein distance) from the population baseline.

2. The Decision Theory Justification (The Barycenter) You argue that the blended curve should be the optimal "center of mass" (barycenter) between the two sources, and the algorithm should find the weight that minimizes the total expected error (Variance + Bias^2) of that final intermediated curve.

Because the bias of a linear barycenter is mathematically defined by the distance between the two source curves, the two paradigms collide. The distance penalty you heuristically enforce in the first approach

organically falls out of the derivative in the second approach.

Ultimately, they both output the exact same solution:

$$\pi(t) = V_{obs}(t) + V_{ext}(t) + (S_{obs}(t) - S_{ext}(t))_2 V_{ext}(t)$$

This means you don't have to choose which mathematical framework is "more correct" for your paper or package vignette—they mutually prove each other.

User prompt: But why are we assuming a linear barycentric

Response: We do not simply assume a linear barycenter out of convenience; it is a deliberate choice driven by three specific statistical and clinical justifications, especially in the context of Health Technology Assessment (HTA).

If you use a linear pool, you are making specific claims about your distance metrics, your hazard assumptions, and your population mixture.

1. The Distance Metric Demands It The shape of a barycenter is not chosen; it is the mathematical output of the distance metric you select to define the "center."

- If you define the information loss using **Reverse KL**, the optimal barycenter is geometric (log-linear).
- If you define the information loss using **Forward KL** or **1-Wasserstein**, the mathematical solution for the barycenter collapses exactly to the arithmetic mean (the linear pool): $S_{blend} = \pi S_{obs} + (1 - \pi) S_{ext}$.

Therefore, by committing to a linear blend, you are implicitly stating that your optimization algorithm penalizes distance according to Wasserstein or Forward KL principles.

2. Avoiding Hazard-Rate Assumptions In survival analysis, the mathematical scale you choose to blend on drastically changes the clinical assumption:

- **Geometric (Log-Linear) Pooling** blends on the log-survival scale. Because the log of survival is the negative cumulative hazard ($\ln(S(t)) = -H(t)$), a geometric barycenter implies you are linearly blending the *hazards* of the two curves.
- **Linear Pooling** blends directly on the probability scale.

Given that dynblend was built explicitly to "smoothly bridge observed trial survival probabilities... without relying on hazard-rate assumptions," the linear barycenter is the only pooling method that operates purely on the survival probabilities without mathematically entangling the hazards.

3. The Population Mixture Interpretation In HTA and decision modeling, a linear combination of probabilities formally represents a **finite mixture model**.

When you use a linear barycenter, the clinical interpretation of $\pi(t)$ is highly intuitive: it represents the probability that a patient's survival mechanism at time t is generated by the trial cohort's dynamics rather than the external population's dynamics. As $\pi(t)$ drops to 0, the mixture smoothly shifts until the cohort is assumed to behave exactly like the background general population. Geometric pooling lacks this clean

demographic interpretation.

4. Mathematical Tractability of Variance To automate $\pi(t)$ by minimizing the Mean Squared Error, you must be able to calculate the variance of the blended curve. The variance of a linear blend is exact and straightforward: $V_{blend}=\pi_2 V_{obs}+(1-\pi)_2 V_{ext}$. Calculating the variance of a geometric blend requires log-transformations and Delta method approximations, which become highly unstable in the tails of the extrapolation where survival approaches 0 (and the log approaches $-\infty$).

User prompt: What about using bayesian posteriors

Response: Transitioning to a fully Bayesian framework is highly advantageous for Health Technology Assessment, as it shifts the output from a single point-estimate curve with standard errors to a full posterior distribution of $S_{blend}(t)$, which plugs directly into Probabilistic Sensitivity Analysis (PSA).

If you possess the Bayesian posteriors for both the trial extrapolation and the external population (e.g., via MCMC draws from Stan or JAGS), the blending mechanics can be approached in three distinct ways.

1. Posterior Predictive Pooling (Sample-by-Sample Blending) Instead of calculating a single weight based on the mean and variance, you apply the linear barycenter equation to every single MCMC iteration independently.

For each posterior draw k:

$$S_{blend(k)}(t)=\pi(t)S_{obs(k)}(t)+(1-\pi(t))S_{ext(k)}(t)$$

You can either keep $\pi(t)$ deterministic (e.g., a pre-specified decay), or you can calculate a dynamic $\pi_{(k)}(t)$ for each draw using the distances between the specific posterior samples. By computing this across thousands of iterations, you generate a complete posterior predictive distribution for the blended curve. This organically captures the exploding uncertainty of the trial extrapolation and correctly propagates it through the blend, yielding robust credible intervals for the economic model.

2. Dynamic Shrinkage (The External Curve as a Prior) Rather than fitting two separate models and blending them post-hoc, you fit a single Bayesian survival model to the trial data where the external population curve serves as a strongly informative, time-varying prior.

In this paradigm, $\pi(t)$ ceases to be an explicit blending weight and instead emerges naturally from the Bayesian updating mechanism:

- **During the trial period:** The likelihood of the observed trial data is highly concentrated. It overwhelms the external prior, so the posterior survival curve tracks the trial data.
- **During the extrapolation period:** The trial data runs out. The likelihood becomes flat and uninformative. By the fundamental mechanics of Bayesian inference, the posterior organically shrinks back toward the prior.

The "blend" happens automatically. The speed of the transition is dictated purely by the precision of the prior versus the diminishing information in the likelihood.

3. Time-Varying Bayesian Model Averaging (BMA) In standard BMA, you calculate the posterior

probability of Model A versus Model B and use those probabilities as weights. If adapted for dynamic blending, $\pi(t)$ becomes the posterior probability that the trial extrapolation is the "true" data-generating mechanism at time t.

$$S_{blend}(t) = P(M_{obs} \mid \text{Data}, t) S_{obs}(t) + P(M_{ext} \mid \text{Data}, t) S_{ext}(t)$$

To make the weights time-varying, the models are evaluated dynamically (often using fractional likelihoods or time-dependent Bayes factors). As the predictive performance of the trial extrapolation degrades in the unobserved future, $P(M_{obs} \mid \text{Data}, t)$ drops toward 0, and the BMA naturally reallocates all probability mass to the external baseline model.

Using Bayesian posteriors eliminates the need for Taylor series approximations of Fisher Information because the posterior distributions inherently contain the exact geometry of the uncertainty and the distance between the curves.

User prompt: Can I get these posterior model probs from kl still

Response: Yes, there is a direct mathematical equivalence between KL divergence and Bayesian posterior model probabilities, bridged by a concept called **Expected Log Predictive Density (ELPD)**.

In modern Bayesian statistics (particularly when using tools like Stan or the loo package in R), you evaluate models based on their out-of-sample predictive accuracy. The fundamental measure of this accuracy is the KL divergence from the true data-generating process to your proposed model.

Information criteria like WAIC or LOO-CV are explicitly designed to estimate this exact KL divergence. Therefore, you can convert KL distance directly into a Bayesian posterior model probability—often referred to as a **Pseudo-BMA weight** or **Akaike weight**.

The Mathematical Conversion To transform a KL distance (which is unbounded) into a posterior model probability $\pi(t)$ (which is strictly bounded between 0 and 1), you pass the negative KL divergence through a softmax function:

$$P(M_{obs} \mid \text{Data}, t) = \exp(-\text{KL}_{obs}(t)) + \exp(-\text{KL}_{ext}(t)) \exp(-\text{KL}_{obs}(t))$$

Because probabilities are relative, it is mathematically identical and computationally more stable to calculate this using the *difference* in KL divergence (ΔKL) between the trial extrapolation and the external curve:

$$\pi(t) = 1 + \exp(\text{KL}_{obs}(t) - \text{KL}_{ext}(t)) 1$$

How this automates $\pi(t)$ dynamically: This equation acts as a logic gate driven by information theory.

- If the trial model captures the survival dynamics at time t better than the external curve, $\text{KL}_{obs}(t) < \text{KL}_{ext}(t)$. The exponent is negative, and $\pi(t)$ shifts toward 1.
- As time progresses and the trial extrapolation's variance explodes, its predictive accuracy plummets. $\text{KL}_{obs}(t)$ becomes much larger than $\text{KL}_{ext}(t)$. The exponent becomes a large positive number, and $\pi(t)$ smoothly collapses to 0.

Implementation in R (dynblend / loo) If you fit your models in a Bayesian framework, you do not even have to calculate the theoretical KL divergence by hand. Bayesian software outputs point-wise log-likelihoods for every observation.

By calculating the point-wise Leave-One-Out Expected Log Predictive Density (LOO-ELPD) at each time *t*, you are computing the exact empirical estimate of the KL divergence. You simply plug those point-wise ELPD values into the softmax equation above (since ELPD ≈ −KL). This approach is known in the Bayesian literature as **dynamic stacking** or **time-varying pseudo-BMA**, and it perfectly operationalises your goal of a fully automated, data-driven survival blend.

User prompt: I don't understand how bma works for different times t

Response: Standard Bayesian Model Averaging (BMA) usually gives you a single, static weight for each model across the entire dataset. To understand how BMA can give you a different weight π at every timepoint *t*, we have to shift from looking at the model's **global** performance to its **local** predictive confidence.

Here is how time-varying BMA (often called Dynamic BMA or Stacking) works, especially in the "unobserved" extrapolation period where your dynblend package operates.

1. The Core Mechanic: Point-Wise Predictive Density

In standard BMA, a model's weight is determined by its Marginal Likelihood—how well it predicted the *entire* dataset.

In time-varying BMA, we slice the timeline into individual moments *t*. Instead of asking "How good is this model overall?", we ask: **"If I pull a random patient out of the true population at time *t*, how tightly did this model predict their exact survival status?"**

This is measured by the point-wise **Expected Log Predictive Density (ELPD)**.

- If a model is highly confident and correct at time *t*, its predictive distribution is tightly packed around the true value. Its ELPD is high.
- If a model is highly uncertain, its predictive distribution spreads its probability mass out very thinly. Even if the mean is correct, the density at any specific point is low. Its ELPD is low.

2. How the Weights Change During Extrapolation

The magic of using this for survival blending happens in the future, beyond the trial data. At year 10, you have no observed patients to test the models against. So how do the weights change?

They change based on the **exploding uncertainty (entropy)** of the extrapolated trial model.

- **The External Registry Model (M_{ext}):** Because it is built on massive population data, its predictive distribution at year 10 is very tight and confident. Its point-wise predictive density remains high and stable.

- **The Trial Extrapolation Model (M_{obs}):** Because it is extrapolating far beyond its data, its parameter uncertainty balloons. By year 10, its predictive distribution for survival is massively wide and flat. It has "smeared" its probability mass over a huge range of possible outcomes.

Because the trial model's probability mass is spread so thin, its Expected Predictive Density drops significantly compared to the external model.

3. Calculating $\pi(t)$

At every timepoint t, the algorithm calculates the expected predictive density for both models and plugs them into the BMA softmax equation:

$$\pi(t)=\exp(\text{ELPD}_{obs}(t))+\exp(\text{ELPD}_{ext}(t))\exp(\text{ELPD}_{obs}(t))$$

The Timeline of the Blend:

1. **At t=1 (Inside the trial):** The trial model has hard data. Its predictive density is very tight and tailored perfectly to the cohort. ELPD_{obs} is much higher than ELPD_{ext}. $\pi(t)$ is close to 1.
2. **At t=5 (Edge of extrapolation):** The trial model's uncertainty starts to grow. Its probability mass begins to flatten out. ELPD_{obs} starts dropping.
3. **At t=10 (Deep extrapolation):** The trial model's variance is huge. Its density is spread incredibly thin. The external registry's tight density now vastly outperforms the trial model mathematically. ELPD_{ext} is much larger than ELPD_{obs}. The equation naturally outputs a $\pi(t)$ near 0.

Why this is mathematically beautiful for dynblend

You do not have to write code that forces the weights to change over time. The formula for the BMA weight remains exactly the same at every timepoint.

The weight $\pi(t)$ changes automatically because the **variance of the trial extrapolation is a function of time**. As time moves forward, the trial's variance expands, its predictive density drops, and the time-varying BMA mathematically demotes it, organically shifting the blend to the external curve.

User prompt: Does the kl related work we talked about previously go into the same paper as the bma idea or are they two different papers

Response: They can absolutely go into one unified paper, and presenting both together creates a highly robust methodological argument. Reviewers with strong technical expertise will appreciate seeing how two entirely different statistical paradigms converge on the exact same optimal solution for survival extrapolation.

- **The Unified Approach:** Framing the manuscript around this mathematical equivalence bulletproofs the methodology. The information-theoretic side (KL/Wasserstein barycenters) provides the clear geometric intuition for automating the blend, while the Bayesian side (ELPD/BMA) provides the probabilistic engine to execute it.
- **The Split Approach (Paper 1 - Analytic):** Focus purely on KL distance, Wasserstein metrics, and automated inverse-variance weights as a rigorous, drop-in upgrade for arbitrary HTA blending

heuristics.
- **The Split Approach (Paper 2 - Bayesian):** Focus strictly on dynamic stacking, MCMC posteriors, and time-varying BMA, specifically targeting how this framework feeds seamlessly into Probabilistic Sensitivity Analysis (PSA).

If the dynblend R codebase currently handles both the analytic calculations and the Bayesian posterior sampling in a single modular workflow, a unified paper makes a fantastic theoretical centrepiece. If the Bayesian implementation requires a much heavier, separate computational architecture, splitting them keeps the primary KL methodology accessible to applied health economists.
